Week 6 Project Progress Report

Adaptive Fabric Surface Defect Detection and Localization using Classical Texture Descriptors

Course Name: ECE501 Digital Image Processing (DIP)
Group Number: Group 15
Group Members: Rinal Shah, Varun Hotani, Aadi Mehta, Pratik Agrawal
Week Number: Week 6
Date: October 27, 2026
Deliverable: Production directory structure; full AFID benchmark; Gabor descriptor; 64×64 refinement; one-class baselines; ablation study
Branch: `feat/week-6` (merged into `main`)

Objectives of the Week

- Reorganise `Codes/` into a production layout taken from published conventions: the PyPA *src layout* [1] and the *cookiecutter-data-science* module split [2]. Each layer (configuration, data, features, modelling, evaluation, pipelines, CLI) gets its own subpackage.
- Acquire the full AFID dataset [3], check it, and re-run the benchmark on *all* defective images. This is the first complete real-data evaluation of the project, after the 13-image mid-semester run and the data-less Week 5.
- Implement the two remaining descriptor and localization items from the original plan: a Gabor filter bank [4] and a nested 64×64 refinement of flagged 256×256 windows.
- Implement the planned baselines, One-Class SVM [6] and Isolation Forest [7]. Then run an ablation over descriptor, scoring, threshold and localization settings, including the *adaptive (per-fabric) vs. fixed (global)* comparison behind the project title.
- Image-level detection, the confidence score and the End-Semester report were *deliberately* left for the final phase.

Summary of the Week

Item	Outcome
Structure	`Codes/src/fabric_defect/` with 8 layer subpackages; tests mirror the package; strictly downward imports
Data	AFID downloaded with a script: 140 defect-free and 106 defective strips; 5 dataset quirks found and handled
Benchmark	All 105 defective strips that have a mask (previously 13). Default patch F1 **18.14 %** (Week-5 setting 13.97 %); pixel IoU **1.36 %** (0.30 %, 4.5×); pixel recall 48.39 % (32.73 %)
Adaptive vs. fixed	Per-fabric reference: patch F1 18.14 % and IoU 1.36 %. One global reference: 5.78 % and 0.07 %. The "adaptive" design is supported by data.
New methods	Gabor bank (24-D), 64×64 refinement (IoU ×1.9), OC-SVM and Isolation Forest baselines, strip-normalised Mahalanobis scoring, pooled metrics
Speed	Per-image feature cache plus 8 worker processes: cold run 163 s, warm run 10 s; 15-variant ablation in 57 s
Key finding	A systematic capture shift between AFID's defect-free and defective strips in 5 of 7 fabrics. It produces 32–100 % false-positive windows there and is the main limit on precision.
Quality	72 automated tests (42 last week), CI on Python 3.11 and 3.13

Work Completed This Week

A. Production Directory Structure

Last week's package was a single flat folder, `Codes/fabric_defect/`, holding 13 modules. Configuration, the CLI, the orchestrator and the leaf algorithms sat side by side. We adopted two widely used conventions:

PyPA src layout [1]. The importable package moves to `Codes/src/`. Python puts the current directory first on the import path, so in a flat layout a test run can silently import the working-tree folder instead of the installed package. With `src/`, the code must be installed (editable for development) before it can be imported. The tests then exercise exactly what a user installs, and files that are not meant to be importable cannot leak onto the path.

cookiecutter-data-science v2 [2]. This template separates a data-science package into `dataset`, `features`, `modeling` (`train`/`predict`) and `plots` modules, and keeps `data/raw` and `data/processed` outside the code. Our package already had these responsibilities, so we turned each into a subpackage. We added `core` (configuration and paths), `pipelines` (end-to-end runs) and `cli`, so that no orchestration or configuration code sits next to the algorithms. Table 1 gives the mapping and Figure 2 shows the layering.

Before (Week 5)

```
Codes/
  pyproject.toml
  configs/default.toml
  fabric_defect/
    __init__.py __main__.py
    cli.py config.py
    afid.py synthetic.py
    preprocessing.py
    features.py model.py
    segmentation.py
    metrics.py pipeline.py
    viz.py
  tests/ (6 flat files)
```

After (Week 6)

```
Codes/
  pyproject.toml # where = ["src"]
  configs/default.toml
  configs/experiments/ablation.toml
  scripts/download_afid.sh
  src/fabric_defect/
    __init__.py __main__.py # entry hook only
    cli/ main.py
    core/ config.py paths.py
    data/ afid.py synthetic.py
    features/ preprocessing.py glcm.py lbp.py
                gabor.py descriptor.py cache.py
    modeling/ reference.py baselines.py
                refinement.py segmentation.py
    evaluation/ metrics.py
    pipelines/ benchmark.py ablation.py
    visualization/ plots.py
  tests/{data,features,modeling,evaluation,
        pipelines,cli}/
```

Figure 1: `Codes/` before and after. Only `__init__.py` and `__main__.py` remain at the package root; Python requires the latter there for `python -m fabric_defect`.

The move was a pure `git mv` with import fixes, committed separately from any behaviour change. Last week's 42 tests passed unchanged from their new mirrored folders. Pytest now runs with `-import-mode=importlib`, because test files in different folders share basenames.

B. AFID Acquisition and Verification

`Codes/scripts/download_afid.sh` fetches the three official archives from the AFID page: `NODefect_images.7z` (94 MB), `Defect_images.7z` (74 MB) and `Mask_images.7z` (86 KB). It extracts them into the git-ignored `data/raw/` and skips archives that are already present. The catalog was then checked file by file. Table 2 gives the composition, and Table 3 lists five quirks of the published archives, each now handled and covered by a test.

Table 1: Mapping from the cookiecutter-data-science module split [2] to our subpackages.

cookiecutter	Ours	Contents
`config.py`	`core/`	Typed, validated dataclass configuration and TOML loading; repository paths
`dataset.py`	`data/` + `scripts/download_afid.sh`	AFID catalog, official defect codes, multi-part masks; download; synthetic smoke-test strips
`features.py`	`features/`	Pre-processing, GLCM, LBP, Gabor, fused descriptor, per-image feature cache
`modeling/train,predict`	`modeling/`	Mahalanobis reference and scoring modes, OC-SVM / Isolation Forest, 64×64 refinement, segmentation
—	`evaluation/`	Patch and pixel metrics, per-image and pooled summaries
`plots.py`	`visualization/`	Inspection collages and charts
`Makefile` targets	`pipelines/` + `cli/`	`run` (benchmark) and `ablate` (grid) subcommands
`data/raw`, `data/processed`	same	AFID images; feature cache (both git-ignored)

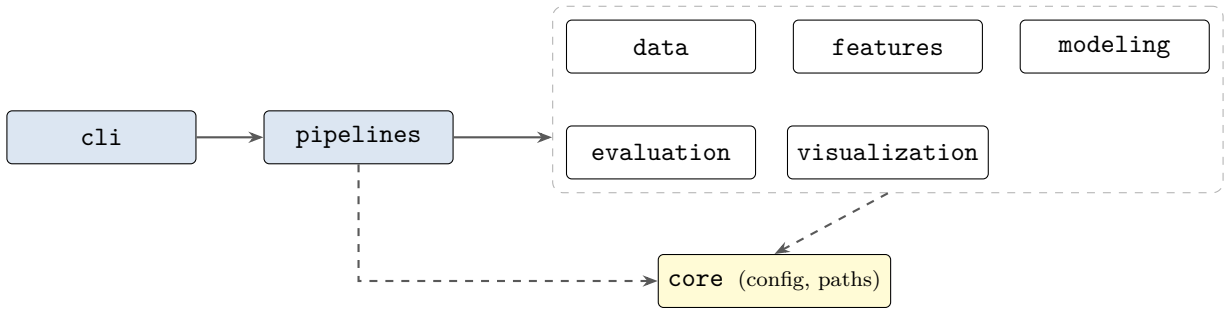


Figure 2: Import direction. `cli` only calls `pipelines`; `pipelines` composes the leaf layers; every layer reads `core`. Nothing imports `cli` or `pipelines`, so each leaf layer is unit-tested in isolation.

The multi-part masks resolve an open question from the mid-semester run. There, `0044_019_04` was evaluated with 0 ground-truth pixels and counted as a miss. The old loader looked only for `<stem>_mask.png` and never found the two parts, which hold 82 defect pixels together. Of the 106 defective strips, 105 have a mask and a reference fabric and are evaluated.

C. Gabor Filter-Bank Descriptor

Gabor filters are band-pass filters that are localized in both space and frequency [5]. A bank of them is a standard texture descriptor [4]. Weaves are periodic, so a filter tuned to the yarn period and orientation responds strongly to intact texture and changes where the period is broken. We use complex filters

$$g_{\lambda,\theta}(x,y) = \exp\Big(-\frac{x'^2+\gamma^2 y'^2}{2\sigma^2}\Big)\exp\Big(i\,\frac{2\pi x'}{\lambda}\Big), \quad x' = x\cos\theta + y\sin\theta,\ y' = -x\sin\theta + y\cos\theta, \quad (1)$$

with the even (real) kernel made zero-mean, so that flat regions respond with 0 rather than with their brightness. The envelope width follows from a 1-octave half-response bandwidth b [5]:

$$\frac{\sigma}{\lambda} = \frac{1}{\pi}\sqrt{\frac{\ln 2}{2}}\,\frac{2^b+1}{2^b-1} = 0.5622 \quad (b=1). \quad (2)$$

Table 2: AFID defective strips by official defect class and fabric code (verified catalog). Every fabric has 20 defect-free strips, split per fabric into 15 for fitting and 5 held out for calibration.

Defect (code)	00	01	02	03	04	05	06	08	All
broken end (002)	3	5	1	0	0	0	0	0	9
broken yarn (006)	0	0	7	0	1	0	0	0	8
broken pick (010)	0	0	0	10	0	0	0	0	10
weft curling (016)	0	0	0	3	0	0	0	0	3
fuzzy ball (019)	0	1	23	5	6	0	4	0	39
cut selvage (022)	4	0	0	3	0	0	2	0	9
crease (023)	0	2	3	0	0	0	0	0	5
warp ball (025)	0	0	0	5	0	0	0	1	6
knot (027)	0	0	0	0	0	1	0	0	1
contamination (029)	0	0	0	0	1	0	0	0	1
nep (030)	0	3	4	5	2	0	0	0	14
weft crack (036)	0	1	0	0	0	0	0	0	1
All	7	12	38	31	10	1	6	1	**106**

Table 3: Quirks found in the published AFID archives and how the pipeline treats them.

File	Quirk	Handling
`NODefect_images/2608691-202020u/0018_00_01.png`	21st defect-free image of fabric 01, with a 2-digit defect field; different content from `0018_000_01`	Skipped with a warning (not a valid `nnnn_ddd_ff` name)
`Defect_images/0100_025_08.png`	Fabric code 08 does not exist, and the image has no mask	Skipped with a warning (no reference, no ground truth)
`0044_019_04`, `0097_030_03`	Mask delivered as `_mask1` + `_mask2`	Parts merged (82 and 81 px)
`0106_010_03`	Mask file is empty	Evaluated; recall-type metrics NaN
`0094_027_05.png`	256×3796 instead of 4096 wide	Supported: no fixed width is assumed

For each window W and filter, the features are the mean and standard deviation of the magnitude $|I * g_{\lambda,\theta}|$ over W, giving $3 \times 4 \times 2 = 24$ values (Table 4). The responses are computed once per strip, so overlapping windows share the convolutions. Unit tests check the selectivity: vertical stripes with an 8-px period excite (λ=8, θ=0°) most, horizontal ones $(8, 90°)$, and a 16-px period $(16, 0°)$.

The descriptor is now assembled from the blocks named in `features.use`: any non-empty subset of `glcm` (30-D), `lbp` (28-D) and `gabor` (24-D). Changing the descriptor is therefore a configuration change, not a code change.

D. Feature Cache and Parallel Extraction

Profiling last week showed that feature extraction dominates run time, while fitting and scoring take milliseconds. Each strip's window features are now computed *once* for every block (GLCM, LBP, Gabor) and every window geometry. They are stored as compressed `.npz` files under `data/processed/` and reused by every run that only changes modelling, scoring or threshold settings. The cache key hashes only the settings that change the numbers (pre-processing, GLCM, LBP and Gabor parameters). Each file is keyed by the image path, size and modification time, so an edited image is recomputed automatically. Files are written atomically (temporary file, then rename), so an interrupted run cannot leave a corrupt entry. LBP codes and Gabor responses are now computed once

Table 4: Gabor bank ($\gamma = 0.5$, orientations $0°, 45°, 90°, 135°$). The wavelengths bracket the 4–8 px AFID yarn period.

λ (px)	σ (px)	kernel size	role
4	2.25	15×15	single-yarn scale
8	4.50	29×29	yarn-pair / weave repeat
16	9.00	55×55	coarse structure (creases, curls)

per strip and then pooled per window. Previously they were recomputed for every overlapping patch, which also means border codes now see their true neighbours. Extraction runs in a standard-library process pool (`-workers`).

Table 5: Wall-clock times on all 246 AFID strips (8 worker processes, laptop CPU).

Run	**Time**
Cold cache, 256×256 windows, all three blocks	60.5 s
Cold cache, 256×256 and 64×64 windows (default)	163.1 s
Warm cache, full default benchmark incl. figures	10.4 s
Warm cache, 15-variant ablation (Section I)	57.4 s

Without the cache, the 15-variant ablation would repeat up to 163 s of extraction per variant (up to about 40 minutes). With the cache it takes under a minute.

E. Nested 64×64 Refinement

A 256×256 window gives stable texture statistics, but when it is flagged all 65,536 of its pixels enter the mask, even for a thread a few pixels wide. That capped pixel IoU near zero in every earlier report. Refinement keeps the coarse decision as a region of interest and re-decides inside it at a finer scale. Let $\mathcal{F}$ be the flagged coarse windows of a strip and

$$R = \bigcup_{w \in \mathcal{F}} w, \qquad \hat{M} = \mathrm{morph}\Big(\bigcup \{\, v : v \subseteq R,\ d_f^{(64)}(\mathbf{x}_v) \geq \tau_f^{(64)} \,\} \Big), \tag{3}$$

where v ranges over the 64×64 windows (stride 32) of the strip, and $d_f^{(64)}$ is the Mahalanobis distance to a *second* reference of fabric f. That reference is fitted on 64×64 windows of the same 15 fit images. Its threshold $\tau_f^{(64)}$ is calibrated on the same 5 held-out images with the same $\mu + k\sigma$ rule ($k = 2.5$). The containment test $v \subseteq R$ uses a summed-area table [8], one $O(1)$ lookup per window. "morph" is the existing 5×5 opening and closing with removal of components under 50 px. Patch-level metrics still use the coarse decision; refinement changes only the pixel mask.

F. One-Class Baselines

To test whether a simple Gaussian model of normal texture is adequate, we added two learned one-class detectors [6, 7]. They run on the same descriptors, the same fit and calibration images and the same threshold rule. Features are standardised on the fit set. The anomaly score is the negated decision function, so that higher means more anomalous, and $\tau = \mu + k\sigma$ on the held-out images. The One-Class SVM uses an RBF kernel with $\nu = 0.1$ and `gamma="scale"`. The Isolation Forest uses 200 trees and is seeded. Only the scoring function differs from the Mahalanobis reference.

G. Adaptive (Per-Fabric) vs. Fixed (Global) Reference

The project's central design choice is *adaptivity*: a separate texture model and threshold for each fabric. `model.reference = "global"` implements the fixed alternative. It pools the fit images of all seven fabrics into one model, pools all calibration images into one τ, and applies both to every strip. The images used are identical in both settings; only the pooling differs.

H. Strip-Normalised Mahalanobis Scoring

The full benchmark revealed a capture shift between defect-free and defective strips (Section "Discussion", Table 10). As a remedy we added `strip_mahalanobis`: the Mahalanobis distances of one strip, converted to one-sided robust z-scores within that strip,

$$s_i = \max\Big(0,\ \frac{d_f(\mathbf{x}_i) - \operatorname{med}_j d_f(\mathbf{x}_j)}{\operatorname{MAD}_j d_f(\mathbf{x}_j) + 10^{-4}}\Big), \qquad \text{flag if } s_i \geq k, \tag{4}$$

which removes any strip-wide offset while keeping the fabric model's notion of what is normal [10].

I. Ablation Protocol and Pooled Metrics

`python -m fabric_defect ablate -grid Codes/configs/experiments/ablation.toml` runs 15 named variants. Each variant changes *one* factor relative to the default (one factor at a time), and all share the same seed. They therefore use the same image-level split, the same 105 test strips and the same cached features, and differences come only from the named factor. The summary reports per-image means (NaN-skipping, as introduced in Week 5) and *pooled* metrics, which sum TP/FP/FN over all strips before forming ratios:

$$\mathrm{IoU}_{\mathrm{pooled}} = \frac{\sum_i TP_i}{\sum_i (TP_i + FP_i + FN_i)}, \qquad F1_{\mathrm{pooled}} = \frac{2\sum_i TP_i}{\sum_i (2\,TP_i + FP_i + FN_i)}. \tag{5}$$

Per-image means give a 15-px warp ball the same weight as a full-width crease. Pooled metrics give every pixel (or patch) the same weight. Both views are reported because they can disagree.

Quantitative Results and Analysis

All numbers in this section are on **real AFID data**: the 105 defective strips that have a mask, with references fitted on 15 and calibrated on 5 defect-free strips per fabric (seed 0). The raw per-strip results are in `Results/final/` and `Results/ablation/`, each with its `run_config.json`.

Headline Result

Table 6: Default configuration vs. the Week-5 setting on all 105 strips (per-image means unless marked pooled).

Setting	Patch F1	Pooled patch F1	Pixel IoU	Pooled pixel IoU	Pixel recall
Week-5 setting: hybrid, GLCM+LBP, no refinement	13.97	14.78	0.30	0.33	32.73
Default: Mahalanobis, GLCM+LBP, 64 × 64 refinement	**18.14**	**22.74**	**1.36**	**1.12**	**48.39**

Ablation

How the default was chosen. A first pass around the Week-5 default (hybrid scoring) favoured all three descriptor blocks and Mahalanobis scoring. Re-centring on Mahalanobis + refinement showed that Gabor's gain belonged to the hybrid scorer. Under Mahalanobis, adding Gabor raises patch recall (63.2 → 72.5 %), but it lowers precision and specificity (55.3 → 42.3 %), patch F1 (18.1 → 17.0 %) and IoU (1.36 → 1.05 %). The default is therefore GLCM+LBP, with Gabor available as an option. GLCM alone is within noise of GLCM+LBP. We keep both, since the patch-level differences are a few strips. $k = 2.0$ scores marginally higher, but it is *not* adopted: picking k on the test strips would bias the result. k stays at the a-priori 2.5.

Table 7: One-factor-at-a-time ablation around the default (all values in %, AFID, $n = 105$). Bold: default. Prec./Rec./Spec. are per-image patch metrics.

Variant	Prec.	Rec.	F1	F1 pool.	Spec.	IoU	IoU pool.	Pix. rec.
default	**18.67**	**63.19**	**18.14**	**22.74**	**55.29**	**1.36**	**1.12**	**48.39**
Descriptor								
GLCM only	19.25	62.97	18.74	22.63	55.32	1.36	1.12	48.17
LBP only	11.81	52.66	11.73	21.58	56.46	1.09	1.10	38.75
Gabor only	13.11	70.65	15.49	22.79	42.44	0.90	1.05	56.47
GLCM+LBP+Gabor	15.25	72.47	16.95	22.85	42.34	1.05	1.01	56.79
Scoring								
strip Mahalanobis	16.72	15.13	9.24	13.33	92.13	0.46	0.86	9.33
hybrid (Week 5)	13.44	32.41	13.97	14.78	75.91	0.72	0.67	27.18
relative	13.41	39.10	14.57	16.57	72.93	0.88	0.90	30.41
Learned baselines								
One-Class SVM	17.78	55.55	16.47	23.41	58.80	1.30	1.11	43.65
Isolation Forest	20.84	48.40	13.96	17.96	64.36	0.85	0.89	39.39
Reference / threshold								
global (fixed) reference	11.46	26.85	5.78	12.03	75.51	0.07	0.10	22.82
$k = 2.0$	18.25	69.77	19.38	23.36	52.09	1.38	1.12	50.74
$k = 3.0$	19.69	59.07	18.40	22.14	57.47	1.34	1.11	46.89
Localization								
refinement off	18.67	63.19	18.14	22.74	55.29	0.70	0.92	53.89
Week-5 setting (all three changes)	13.44	32.41	13.97	14.78	75.91	0.30	0.33	32.73

Per-Fabric and Per-Defect Results (Default)

Table 8: Default configuration per fabric (%, per-image means), with the capture-shift statistics of Table 10 for comparison.

Fabric	n	Patch prec.	Patch rec.	Patch F1	Pixel IoU	Pixel rec.	FP rate on normal windows
fabric_00	7	6.90	57.14	9.07	0.06	57.14	57.33
fabric_01	12	14.73	100.00	20.89	1.93	89.15	93.63
fabric_02	38	4.09	75.00	7.42	0.09	55.26	53.83
fabric_03	31	**39.95**	63.15	**34.30**	**3.73**	39.08	31.94
fabric_04	10	0.00	0.00	0.00	0.00	0.00	1.72
fabric_05	1	6.90	100.00	12.90	0.03	100.00	100.00
fabric_06	6	25.00	30.00	25.00	0.07	31.67	9.09

The AFID Capture Shift

Discussion of Findings

1. **Adaptivity is justified by data.** Replacing the per-fabric references and thresholds with one global model is the largest single degradation in the ablation. Patch F1 falls from 18.14 to 5.78 %, pixel IoU from 1.36 to 0.07 % and pixel recall from 48.4 to 22.8 %. The seven weaves occupy different regions of descriptor space, so a single covariance spans all of them, and a pooled τ is too loose for some fabrics and too tight for others.
2. **The capture shift sets the false-positive floor.** On held-out *defect-free* strips the thresh-

Table 9: Default configuration per official defect class (%, per-image means). "–" = undefined: no window of any of these strips holds more than 25 mask pixels.

Defect	n	Patch prec.	Patch rec.	Patch F1	Pixel IoU	Pixel rec.
broken pick (010)	10	76.00	57.28	50.65	10.91	47.76
weft crack (036)	1	93.55	100.00	96.67	22.39	98.83
cut selvage (022)	9	51.14	44.44	36.83	0.21	43.34
weft curling (016)	3	15.48	50.00	23.61	1.57	47.53
crease (023)	5	8.57	60.00	14.54	0.49	80.00
broken end (002)	9	6.53	88.89	12.14	0.11	85.66
knot (027)	1	6.90	100.00	12.90	0.03	100.00
broken yarn (006)	8	6.91	87.50	11.30	0.12	87.50
fuzzy ball (019)	39	7.70	50.00	9.42	0.01	25.64
nep (030)	14	4.73	69.23	8.79	0.02	64.29
warp ball (025)	5	0.00	–	0.00	0.02	20.00
contamination (029)	1	0.00	0.00	0.00	0.00	0.00

Table 10: Default-config Mahalanobis distance (256×256) per fabric: held-out *defect-free* calibration strips vs. the *normal* windows (no defect pixels above the 25-px rule) of the defective test strips.

Fabric	τ_f	Calib. median	Calib. FP rate (%)	Test-normal windows	Test-normal median	Test-normal FP rate (%)
fabric_00	4.03	2.44	0.93	150	6.96	57.33
fabric_01	4.44	2.50	2.90	314	12.95	93.63
fabric_02	4.58	2.45	1.38	1,098	12.82	53.83
fabric_03	4.42	2.72	1.74	551	3.54	31.94
fabric_04	4.55	2.35	1.29	291	2.04	1.72
fabric_05	4.20	2.29	2.76	27	23.09	100.00
fabric_06	3.71	2.25	3.23	176	2.45	9.09

olds behave exactly as calibrated (0.9–3.2 % false positives, Table 10). On defect-free *windows of defective strips* the rate is 32–100 % for fabrics 00, 01, 02, 03 and 05, whose median distances are 1.3–10× the calibration median. Only fabrics 04 and 06 transfer (1.7 % and 9.1 %). This matches the per-fabric precision in Table 8, and Figure 4 shows the effect. AFID's defect-free and defective images appear to come from different acquisitions (rolls, tension or lighting), a dataset shift in the sense of [9]. It explains why the Week-5 strip-relative scores were introduced. It is also why every scorer, learned or not, stays below about 20 % patch precision: the scorers differ in how they trade recall for specificity, not in whether they see the shift.

3. **Strip normalisation buys specificity, not accuracy.** `strip_mahalanobis` removes the offset (specificity 92.1 %) but also suppresses defects that span many windows, so recall drops to 15 %. It is a useful high-precision operating point, not a better default. A correct fix needs reference data from the same acquisition as the test strips (see Plan).

4. **Refinement doubles localization quality.** Refinement leaves patch decisions unchanged by design, and it raises mean pixel IoU from 0.70 to 1.36 % and pooled IoU from 0.92 to 1.12 %. In return, pixel recall drops from 53.9 to 48.4 % where a fine window falls just below $\tau^{(64)}$. Where the coarse decision is right, the effect is large (Figure 3: 63 % IoU). Where the capture shift has flagged everything, refinement cannot help, because the fine model shifts too.

5. **GLCM carries the descriptor.** GLCM alone matches GLCM+LBP. LBP alone is weakest (F1 11.7 %), as its histograms change little for thin defects inside a 256×256 window. Gabor alone reaches the highest recall of any descriptor (70.7 %), but at the lowest specificity. The tuned band-pass responses react to the shift as well as to defects.

6. **The Gaussian model is competitive with learned one-class methods.** The One-Class

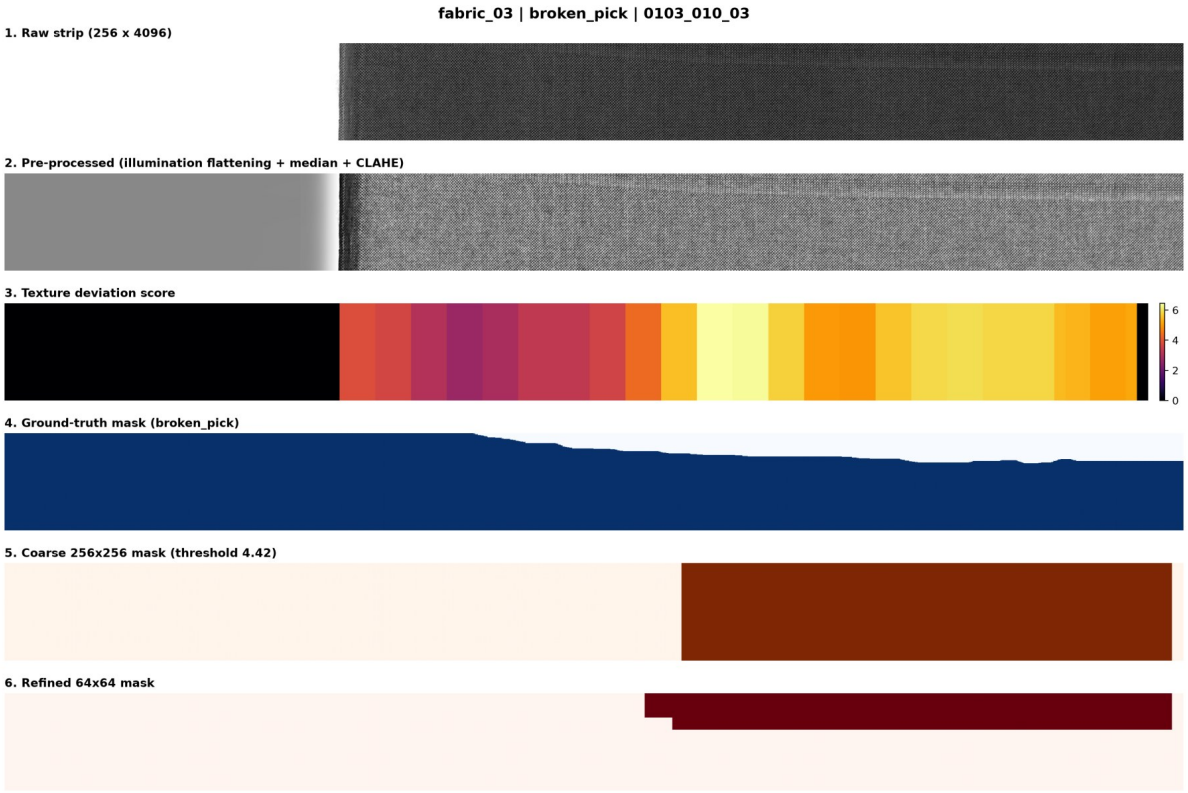


Figure 3: Success case, AFID `0103_010_03` (broken pick, fabric_03): pixel IoU 63.3 %, pixel recall 85.5 %. The 256×256 decision (panel 5) flags the right half of the strip. The 64×64 refinement (panel 6) reduces it to the band along the top rows where the pick is missing (white region in panel 4).

SVM edges the default on pooled patch F1 (23.4 vs. 22.7 %). It is below on mean F1 (16.5 vs. 18.1 %), IoU (1.30 vs. 1.36 %) and recall. The Isolation Forest is clearly weaker (F1 14.0 %). With 15 fit images per fabric, a regularised mean and covariance are as good a description of normal texture as these learned boundaries, and far cheaper.

7. **Defect classes split by geometry.** Weft-direction defects that span the strip width are found and localised best: broken pick (F1 50.7 %, IoU 10.9 %), weft crack and cut selvage. Small, compact defects are the hardest: fuzzy ball ($n = 39$, IoU 0.01 %), nep and warp ball. All five warp-ball masks have only 11–24 pixels, below the 25-pixel patch-label rule, so their patch recall is undefined rather than zero. Fabric_04 is the extreme case. It has almost no capture shift, but its defects are all small (6 fuzzy balls, 2 neps, 1 broken yarn, 1 contamination) and none is detected.

8. **Absolute levels remain low.** Even the best configuration localises about 1 % of defect pixels correctly by IoU. The improvements are real and reproducible (pixel IoU $\times$4.5, recall +15.7 points over the Week-5 setting), but the system is far from usable inspection accuracy. The capture shift and small defects are the two bottlenecks, and both are now quantified.

Testing and Continuous Integration

The suite grew from 42 to 72 tests (about 2 minutes, including synthetic end-to-end runs of both subcommands), mirrored into `tests/\{data, features, modeling, evaluation, pipelines, cli\}/`. New tests cover the following.

- **Data.** The malformed AFID filename is skipped.
- **Gabor.** One-octave σ (Eq. 2); orientation and wavelength selectivity on synthetic stripes; zero response on flat images; block selection in the descriptor.

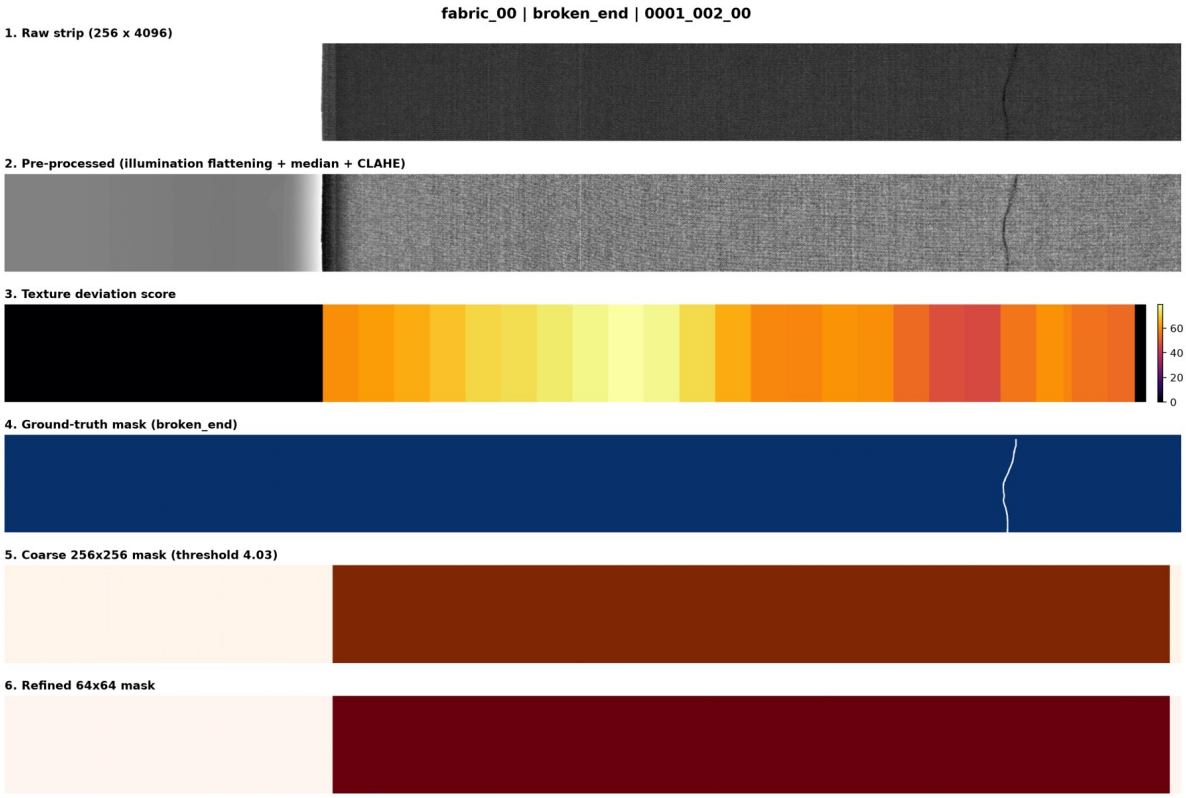


Figure 4: Failure case caused by the capture shift, AFID `0001_002_00` (broken end, fabric_00). Every window of the strip scores 40–70 against $\tau = 4.03$ (panel 3), so both the coarse and the refined mask cover the whole fabric. The defect (panel 4) is found, but precision is near zero.

- **Cache.** ROI-relative coordinates; round trip through the cache with 1 vs. 2 workers (bit-identical); separate cache entries when feature settings change; reuse when only modelling settings change; per-scale reuse.
- **Modelling.** Refinement region, containment by summed-area table (vs. brute force) and the refined mask; OC-SVM and Isolation Forest rank outliers above normal data and are seeded; strip normalisation removes a global offset; the global reference shares one model across fabrics.
- **Evaluation.** Pooled vs. mean pixel IoU on a two-image example.
- **Pipelines and CLI.** Grid parsing, duplicate or unknown keys rejected, `ablate` end-to-end through the CLI, exit code 2 for a bad grid.

GitHub Actions runs the suite on Python 3.11 and 3.13 on every push and pull request.

Commit History

Table 11: Commits of the week on `feat/week-6`, oldest first (single-purpose, one-line messages).

Hash	Message
`bf91e67`	refactor: move package to src layout with layered subpackages
`242935c`	chore: add AFID download script and skip malformed AFID filenames
`42fc4d5`	feat: add Gabor filter-bank descriptor and configurable feature blocks
`c994611`	feat: cache window features per image and extract them in parallel
`5c97f1f`	feat: refine flagged patches at 64x64 against a fine-scale reference
`6e32ebf`	feat: add One-Class SVM and Isolation Forest baselines

Hash	Message
`ef323e8`	feat: add ablation runner and a global reference for fixed-vs-adaptive comparison
`a86fd92`	feat: report pooled patch and pixel metrics alongside per-image means
`5fe1048`	feat: add strip-normalised Mahalanobis scoring for the AFID capture shift
`6dd5f82`	feat: default to Mahalanobis on GLCM+LBP+Gabor with 64x64 refinement
`5d2fcd6`	fix: keep GLCM+LBP as the default descriptor after the Mahalanobis ablation
`3b8b940`	chore: publish full AFID benchmark and ablation results and archive legacy outputs
`7f24fdc`	docs: update README for src layout, AFID download and Week 6 results

The two default-changing commits are kept as they happened. Commit `6dd5f82` adopted the first-pass winner; `5d2fcd6` reverted the descriptor part after the re-centred ablation (Section "Ablation").

Literature Review

Topic / Paper	Key Takeaway	Relevance to the Project
src layout vs. flat layout (PyPA) [1]	Importable code under `src/` must be installed before it can be imported; tests then run against the installed package and stray files cannot shadow it.	Basis of the new `Codes/src/` layout and the editable install used in development and CI.
Cookiecutter Data Science [2]	A conventional project tree: raw and processed data outside the code; dataset, features, modelling and plotting as separate modules.	Basis of the subpackage split (Table 1) and of `data/processed` as the feature-cache location.
Unsupervised texture segmentation using Gabor filters [4]	A bank of Gabor filters at several frequencies and orientations, with local energy statistics, separates textures.	The 24-D Gabor block (Eq. 1, Table 4).
Uncertainty relation for 2-D visual filters [5]	Gabor functions achieve the joint space–frequency resolution limit; bandwidth sets σ/λ.	The one-octave $\sigma = 0.5622\lambda$ rule (Eq. 2).
Estimating the support of a high-dimensional distribution [6]	One-Class SVM: a kernel boundary around the training data with outlier fraction bounded by ν.	Learned baseline; nearly matches but does not beat the Mahalanobis reference.
Isolation Forest [7]	Anomalies are isolated by fewer random splits; linear-time, no density estimate.	Second baseline; weaker than both other scorers on AFID.
Summed-area tables [8]	Any rectangle sum in $O(1)$ after one cumulative pass.	Fast "fine window inside coarse region" test in refinement (Eq. 3).
Dataset Shift in Machine Learning [9]	Training and test distributions can differ; models calibrated on one can fail on the other.	Frames the AFID capture shift (Table 10) as the main open problem.
AFID [3]	Public benchmark: 7 fabrics, 12 defect classes, pixel masks.	Dataset; checked and its quirks documented (Tables 2, 3).
GLCM [11]; LBP [12]	Co-occurrence statistics; rotation-invariant uniform local binary patterns.	The default GLCM+LBP descriptor; GLCM shown to carry most of the signal.

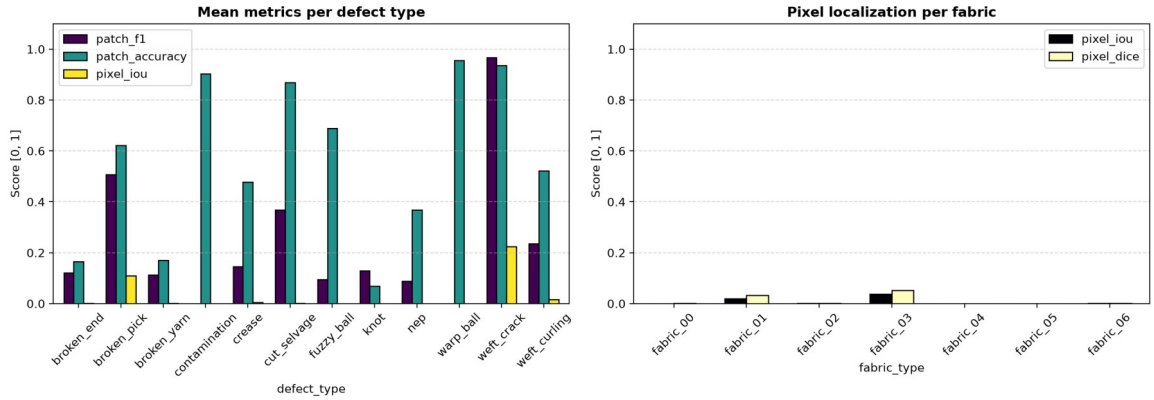


Figure 5: Mean metrics per official defect class and per fabric for the default configuration (from `Results/final/figures/`).

Challenges Faced

- **Dataset quirks.** The archives contain an extra, misnamed defect-free image, a defective image of a non-existent fabric without a mask, two split masks, one empty mask and one shorter strip (Table 3). Each now has explicit handling and a test, rather than silently changing the counts.
- **Capture shift.** The largest effect in the results is a property of the data, not of any method. It took a dedicated measurement (Table 10) to separate it from descriptor and scorer choices, and it limits what any defect-free-trained detector can reach on AFID.
- **Avoiding test-set tuning.** With 105 strips and no separate validation set of defective images, every default choice risks fitting the test set. We fixed $k = 2.5$ a priori, chose defaults only between methods (not fine parameter values), and report the full ablation so the reader can see every alternative.
- **Run time.** The first full runs took minutes per configuration. The per-image cache made iterating over 15+ variants practical (Table 5).

Plan for Next Week (Week 7)

- **Image-level detection and confidence score.** Add the 35 held-out defect-free strips to the test set to measure image-level false alarms, which the defective-only protocol cannot measure. Derive a per-image confidence from the strip's maximum normalised distance.
- **Address the capture shift.** Calibrate on defect-free regions of defective strips using a strip-level split that never shares a strip between calibration and testing. Also evaluate a per-strip offset correction applied only to the background distribution, which keeps the recall that full strip normalisation loses.
- **Small defects.** Evaluate a 32×32 refinement level and a lower patch-label rule for fuzzy balls, neps and warp balls (11–24 px masks).
- **End-semester preparation.** Freeze the protocol, then collect final figures and tables for the End-Semester report and presentation.

Conclusion

Week 6 delivered four things:

- a production project structure, following the PyPA src layout and the cookiecutter-data-science module split
- the first complete evaluation on real AFID data
- the two outstanding method components: a Gabor filter bank and nested 64×64 refinement
- two learned baselines and a reproducible 15-variant ablation

On all 105 masked defective strips, the new default (per-fabric Mahalanobis on GLCM+LBP with refinement) improves on the Week-5 setting in patch F1 (13.97 → 18.14 %), pixel IoU (0.30 → 1.36 %) and pixel recall (32.7 → 48.4 %). The ablation supports the project's adaptive design: a single global model collapses to 5.8 % F1. The dominant remaining error is not a modelling choice but a measurable shift between AFID's defect-free and defective captures. Together with very small defects, it defines the work left for the final phase.

References

[1] Python Packaging Authority, "src layout vs flat layout," *Python Packaging User Guide*. [Online]. Available: https://packaging.python.org/en/latest/discussions/src-layout-vs-flat-layout/

[2] DrivenData, "Cookiecutter Data Science," v2. [Online]. Available: https://cookiecutter-data-science.drivendata.org/

[3] J. Silvestre-Blanes, T. Albero-Albero, I. Miralles, R. Pérez-Llorens, and J. Moreno, "A public fabric database for defect detection methods and results," *AUTEX Res. J.*, vol. 19, no. 4, pp. 363–374, Dec. 2019, doi: 10.2478/aut-2019-0035.

[4] A. K. Jain and F. Farrokhnia, "Unsupervised texture segmentation using Gabor filters," *Pattern Recognit.*, vol. 24, no. 12, pp. 1167–1186, 1991, doi: 10.1016/0031-3203(91)90143-S.

[5] J. G. Daugman, "Uncertainty relation for resolution in space, spatial frequency, and orientation optimized by two-dimensional visual cortical filters," *J. Opt. Soc. Amer. A*, vol. 2, no. 7, pp. 1160–1169, 1985.

[6] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, Jul. 2001, doi: 10.1162/089976601750264965.

[7] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[8] F. C. Crow, "Summed-area tables for texture mapping," *Comput. Graph. (Proc. SIGGRAPH)*, vol. 18, no. 3, pp. 207–212, 1984.

[9] J. Quiñonero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, Eds., *Dataset Shift in Machine Learning*. Cambridge, MA, USA: MIT Press, 2009.

[10] P. J. Rousseeuw and C. Croux, "Alternatives to the median absolute deviation," *J. Amer. Statist. Assoc.*, vol. 88, no. 424, pp. 1273–1283, 1993, doi: 10.1080/01621459.1993.10476408.

[11] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-3, no. 6, pp. 610–621, Nov. 1973, doi: 10.1109/TSMC.1973.4309314.

[12] T. Ojala, M. Pietikäinen, and T. Mäenpää, "Multiresolution gray-scale and rotation invariant texture classification with local binary patterns," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 971–987, Jul. 2002, doi: 10.1109/TPAMI.2002.1017623.